SOFTWARE REQUIREMENTS SPECIFICATION

SMART CALENDAR APPLICATIONS

Project Documentation

Document Version	1.0
Status	Academic project draft
Prepared Date	9 October 2026
Prepared By	Student Project Team

This document describes the proposed features, interfaces, constraints, and quality requirements for a Smart Calendar Applications project. Update implementation-specific details to match your final design.

Document Control

Version	Date	Description
1.0	09-Oct-2026	Initial SRS draft for academic project

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define what the Smart Calendar Applications system is expected to do. It is intended for the student developer, project guide, testers, and users reviewing the proposed system.

1.2 Scope

Smart Calendar Applications is a proposed digital calendar that helps users organize events, tasks, reminders, and schedules in one place. The system is intended to make planning easier, reduce missed activities, and let users view upcoming commitments by day, week, or month.

- Create, view, edit, and delete calendar events.
- Add tasks with due dates, priorities, and completion status.
- Set reminders for events and tasks.
- Search and filter calendar entries.
- View schedules in daily, weekly, and monthly formats.
- Manage basic user profile and preferences, if authentication is included in the implementation.

1.3 Definitions and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
Event	A scheduled activity with a date and time
Task	An item that needs to be completed
Reminder	A notification or alert before an event or due date
UI	User Interface
DB	Database

1.4 References

The document structure is informed by common SRS practices and the requirements-engineering guidance in ISO/IEC/IEEE 29148:2018. This student document is a tailored project draft, not a claim of formal standards compliance.

1.5 Document Overview

The remaining sections describe the product context, functional and non-functional requirements, interfaces, data requirements, constraints, assumptions, and verification plan.

2. Overall Description

2.1 Product Perspective

The application is a standalone calendar system. Depending on the final implementation, it may run as a web application, desktop application, or mobile application. Persistent storage is required if events and tasks must remain available after the application closes.

2.2 Product Functions

- Calendar navigation by date and view.
- Event and task management.
- Reminder creation and delivery through supported notification methods.
- Search, sorting, and filtering of entries.
- Input validation and helpful error messages.
- Optional sign-in and profile preferences, if included in the project scope.

2.3 User Classes

- Registered user: manages personal events, tasks, and preferences.
- Guest user (optional): may view or create temporary entries if the implementation supports guest mode.
- Administrator (optional): may manage application-level settings only if an administrator role is implemented.

2.4 Operating Environment

The target environment will depend on the chosen platform. A web version requires a supported modern browser; a desktop version requires a compatible operating system; a mobile version requires a supported mobile OS. The application needs access to local or server-side storage. Notifications may require device or browser permission.

2.5 Design and Implementation Constraints

- The project must use the programming language, framework, and database selected by the student or project guide.
- The user interface should remain understandable for users with basic computer or smartphone skills.
- The system must validate required fields and date/time values.
- Reminder delivery depends on platform permissions and whether the application or its notification service is running.
- Features such as cloud synchronization, sharing, and calendar import/export are outside the core scope unless explicitly implemented.

2.6 Assumptions and Dependencies

- The user enters accurate event details and selects the correct date and time.
- The device clock and time zone are configured correctly.
- The selected storage technology is available and functioning.
- If online synchronization is implemented, a network connection and server service are available.

3. System Features and Functional Requirements

Each requirement has an identifier so it can be checked during development and testing. Requirements marked optional should be retained only if the final implementation includes that feature.

FR-01: Open calendar

The system shall display a calendar and allow the user to move to a selected date.

FR-02: Change view

The system shall allow the user to switch between day, week, and month views, where supported.

FR-03: Create event

The system shall allow the user to enter an event title, date, start time, end time, and optional description or location.

FR-04: Validate event

The system shall reject an event with a missing required title/date or an invalid time range and shall explain the error.

FR-05: View event

The system shall display saved event details when the user selects an event.

FR-06: Edit event

The system shall allow the user to modify an existing event and save the changes.

FR-07: Delete event

The system shall allow the user to delete an event and request confirmation before permanent deletion.

FR-08: Create task

The system shall allow the user to create a task with a title, due date, priority, and completion status.

FR-09: Update task status

The system shall allow the user to mark a task as complete or incomplete.

FR-10: Set reminder

The system shall allow the user to set a reminder time for an event or task.

FR-11: Deliver reminder

The system shall show or send a reminder using a notification method supported by the selected platform and granted permissions.

FR-12: Search entries

The system shall allow the user to search events and tasks by title or keyword.

FR-13: Filter entries

The system shall allow the user to filter entries by date range or task status, where supported.

FR-14: Save data

The system shall store created and updated entries so they remain available after the application is closed and reopened.

FR-15: User preferences (optional)

If preferences are implemented, the system shall allow the user to change supported settings such as theme or default calendar view.

FR-16: Account access (optional)

If user accounts are implemented, the system shall support sign-in/sign-out and restrict each user to their own private entries.

4. External Interface Requirements

4.1 User Interface

- The home screen should show the selected date and calendar entries clearly.
- Primary actions should be labelled: Add Event, Add Task, Search, Save, Edit, and Delete as applicable.
- Forms should identify required fields and show understandable validation messages.
- The interface should use readable text, consistent navigation, and visible confirmation after saving or deleting an entry.

4.2 Hardware Interface

No special hardware is required beyond the device on which the application runs. Notification capability depends on the device and platform.

4.3 Software Interface

- A database or local storage component shall store calendar events, tasks, and supported preferences.
- The application shall use the selected platform's supported APIs or libraries for date/time handling and notifications.
- If authentication or cloud sync is implemented, the application shall communicate with the relevant authentication and server services.

4.4 Communication Interface

Internet access is not required for a fully local version. It is required for features such as cloud synchronization, remote account access, or server-backed notifications.

5. Data Requirements

5.1 Main Data Entities

Entity	Example fields	Purpose
Event	event_id, title, date, start_time, end_time, description, location, reminder_time	Stores scheduled activities
Task	task_id, title, due_date, priority, status, reminder_time	Stores work to be completed
User (optional)	user_id, display_name, email, password_hash or identity_provider_id, preferences	Stores account and preference data if accounts exist

5.2 Data Integrity

- Each event and task shall have a unique identifier.
- Required fields shall be validated before saving.
- Changes shall be saved consistently; the application should not silently discard valid user edits.
- Dates and times shall be stored and displayed consistently with the chosen time-zone approach.

5.3 Privacy and Retention

Calendar entries can contain personal information. The application should only expose a user's entries to that user unless sharing is deliberately implemented. If accounts are included, passwords must not be stored as plain text. The project should explain how local or server-stored data can be removed.

6. Non-Functional Requirements

NFR-01: Usability

Common actions such as creating an event should be achievable with a small number of clear steps.

NFR-02: Performance

For a typical student-project dataset on supported hardware, common actions should normally respond within 2 seconds, excluding network delays.

NFR-03: Reliability

The application should handle invalid input without crashing and should preserve successfully saved data.

NFR-04: Security

If accounts or remote services are used, access shall be authenticated and private data shall be protected in transit and at rest where supported.

NFR-05: Compatibility

The application shall run on the platform and versions specified in the final project report.

NFR-06: Maintainability

The code should be divided into understandable modules for calendar display, event/task handling, storage, and reminders where practical.

NFR-07: Accessibility

Controls should have clear labels and the interface should maintain readable contrast and text size.

NFR-08: Backup/recovery

If cloud storage or export is implemented, the project should provide a documented recovery or export method.

7. Use Case Summary

Actor	Use case	Outcome
User	View calendar	Selected schedule is displayed
User	Create/edit/delete event	Event data is saved or removed
User	Manage tasks	Task details and status are updated
User	Set reminder	Reminder is scheduled if supported
User	Search/filter entries	Matching entries are displayed
User	Manage preferences (optional)	Selected preferences are applied

8. Verification and Acceptance Criteria

The project can be considered ready for demonstration when the implemented requirements below have been checked. Record actual test results in a separate test report.

ID	Test	Expected result
TC-01	Create an event with valid details	Event appears on the selected date and remains after reopening.
TC-02	Submit an event without a required field	System blocks saving and displays a clear message.
TC-03	Edit an existing event	Updated details are displayed after saving.
TC-04	Delete an event	Confirmation is shown and the event is removed after confirmation.
TC-05	Create and complete a task	Task appears in the task list and completion status changes.
TC-06	Set a reminder with notifications permitted	A reminder is shown at the configured time, subject to platform limits.
TC-07	Search for a known entry	Matching events/tasks are displayed.
TC-08	Close and reopen the app	Previously saved entries remain available.

9. Risks and Limitations

- Notifications may be delayed or unavailable because of device settings, permissions, or operating-system restrictions.
- The final features depend on the selected technology stack and the time available for implementation.
- Cloud sync, account management, shared calendars, and external calendar integration are optional and should not be claimed unless implemented and tested.
- The SRS describes intended behavior; it does not prove that the software has already been built.

10. Future Enhancements

- Recurring events and repeat reminders.
- Calendar sharing and collaboration.
- Cloud backup and synchronization across devices.
- Import/export to common calendar formats.
- Smart scheduling suggestions based on user-selected priorities.

11. Conclusion

This SRS provides a baseline for developing and testing Smart Calendar Applications. Before submission, the student should replace generic platform details with the actual language, framework, database, and features used in the implementation, and remove optional features that are not part of the project.

Appendix A: Quick Requirement Checklist

■ Calendar day/week/month navigation

- Event create, view, edit, and delete
- Task create and completion status
- Reminder creation and delivery
- Search and filtering
- Persistent data storage
- Validation and error messages
- Basic privacy and security checks where applicable
- Testing evidence for implemented features